

Telefon Sensörlerinden Konum Bulma

GPS + İvmeölçer Füzyonu ve Kalman Filtresi — Eğitim Notu

Başlangıç Sorusu

“Elimizde sadece bir telefon var. Trenin ya da aracın nerede olduğunu ne kadar doğru bulabiliriz?”

Telefonunuzda birbirini tamamlayan iki tür sensör vardır:

- **GPS** → nerede olduğunuzu doğrudan söyler; ancak kabadır (5–10 m), tünelde/kapalı alanda kaybolur ve saniyede yalnızca 1 kez gelir.
- **İvmeölçer + jiroskop** → çok hızlıdır (saniyede 50–100 ölçüm) ve hiç kesilmez; ama konumu doğrudan vermez, yalnızca *“ne kadar hızlandın, ne kadar döndün”* der.

Bu eğitim notunun konusu: **Bu iki sensörü nasıl akıllıca birleştiririz?**

Ders 1 — İvmeölçer Tek Başına Neden Konum Veremez?

İvmeölçer **ivmeyi** ölçer. Konuma ulaşmak için arka arkaya iki kez integral almak gerekir:

ivme → (1 integral) → hız → (1 integral daha) → konum

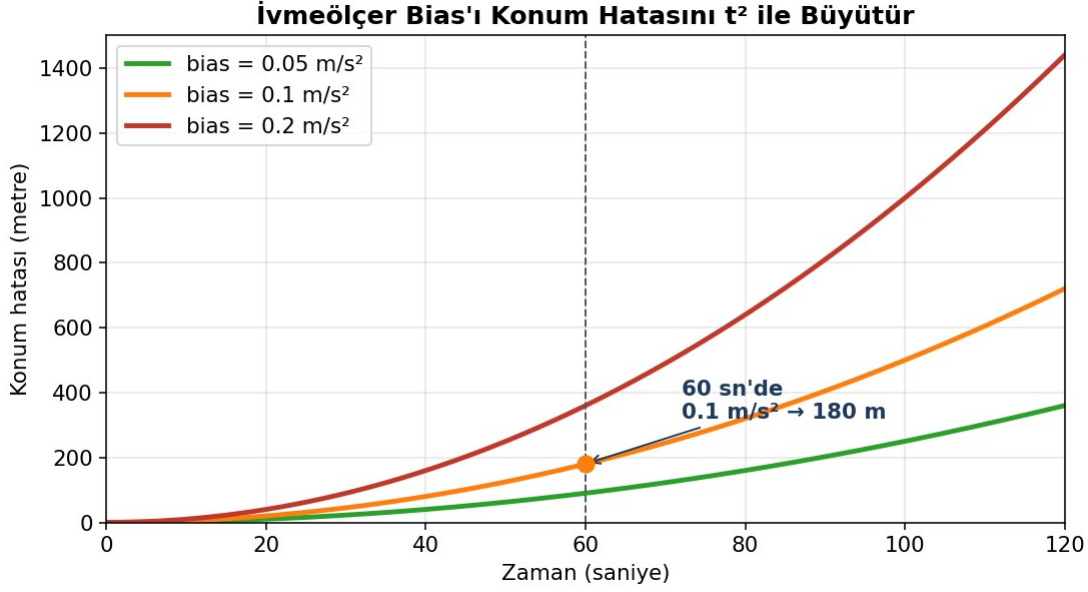
Sorun: Sensörde minik bir sabit hata (*bias*) bile bulunur ve bu hata integral aldıkça büyür:

Büyüklik	Hataya etkisi
Hızda	$\text{bias} \times t$ (zamanla doğrusal büyür)
Konumda	$\frac{1}{2} \times \text{bias} \times t^2$ (zamanla karesel büyür!)

Sayısal örnek (tahtada gösterilebilir): Telefonda tipik $\text{bias} = 0.1 \text{ m/s}^2$. 60 saniye sonra konum hatası:

$$\frac{1}{2} \times 0.1 \times 60^2 = 180 \text{ metre}$$

Aşağıdaki grafik, üç farklı bias değeri için hatanın nasıl patlayarak büyüdüğünü gösterir. Dikkat edin: hata zamanla *düz çizgi gibi* değil, *yukarı kıvrılan bir eğri* gibi büyüyor — işte karesel (t^2) büyüme budur.



İvmeölçer bias'ı konum hatasını t^2 ile büyütür

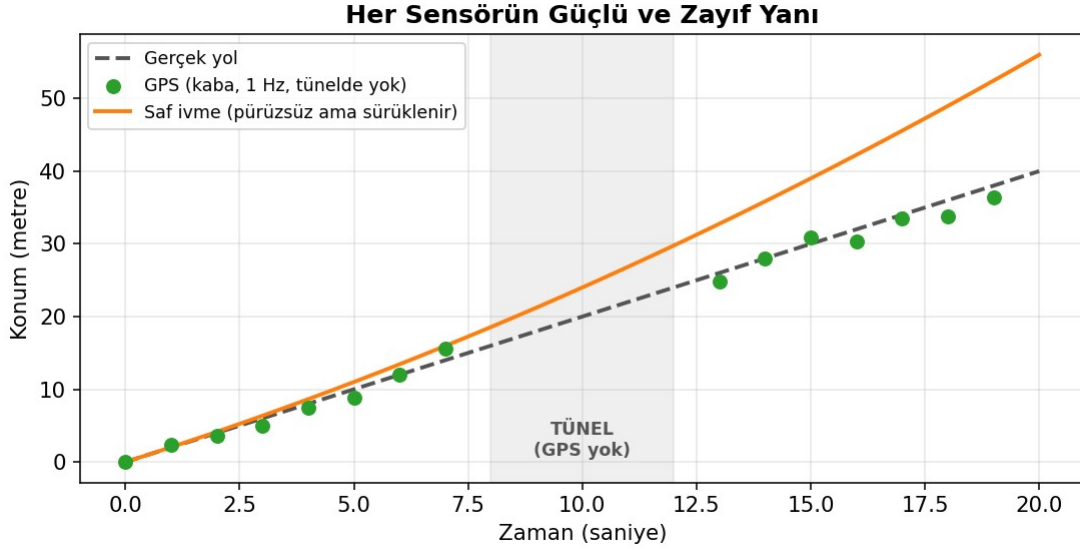
💡 **Ders çıkarımı:** “Saf ivmeölçer ile navigasyon” fiziksel olarak imkânsızdır. Drift (sürüklenme) kaçınılmazdır.

Ders 2 — Çözüm: Kalman Filtresi ile “Füzyon”

Fikir basit ve zariftir: **İki sensörün güçlü yanlarını birleştir.**

- **İvmeölçer:** GPS örnekleri *arasını* doldurur (hızlı, sürekli).
- **GPS:** ivmeölçerin drift'ini sürekli *düzeltilir* (doğru ama yavaş).

Aşağıdaki grafik bu tamamlayıcılığı gösterir: GPS noktaları (yeşil) seyrek ve gürültülüdür, tünelde hiç yoktur; saf ivme (turuncu) ise pürüzsüzdür ama gerçek yoldan (kesik çizgi) giderek uzaklaşır.

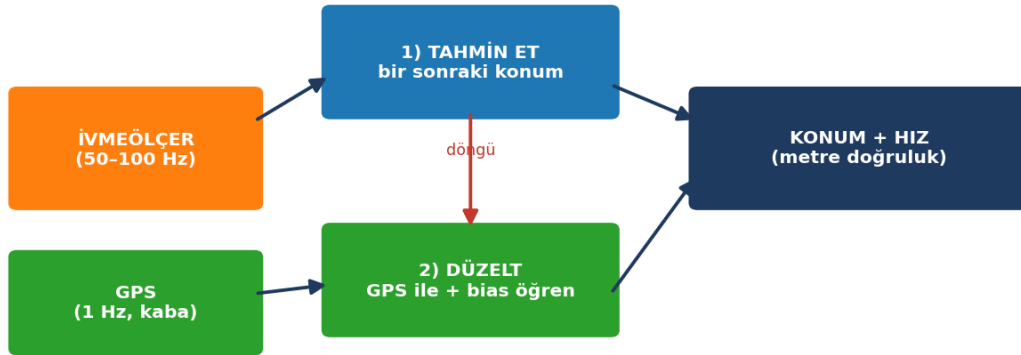


Her sensörün güçlü ve zayıf yanı

Kalman filtresi tam olarak bu birleştirmeyi yapan matematiksel araçtır. Her an **iki adımda** çalışır:

1. **TAHMİN ET:** İvmeölçerle "bir sonraki an neredeyim" diye tahmin yürüt
2. **DÜZELT:** GPS gelince "aslında buradaymışım" diye tahmini düzelt + bu arada sensörün bias'ını da ÖĞREN ve çıkar!

Kalman Filtresi: Tahmin Et → Düzelt Döngüsü



Kalman filtresi: tahmin et → düzelt döngüsü

Bizim filtremiz aynı anda **6 şeyi** takip eder: konum (Kuzey-Doğu), hız (Kuzey-Doğu) ve **ivmeölçerin bias'ı**. Bias'ın canlı öğrenilip çıkarılması, Ders 1'deki 180 metrelik felaketi engelleyen kilit noktadır.

Ders 3 — Deneyle İspat

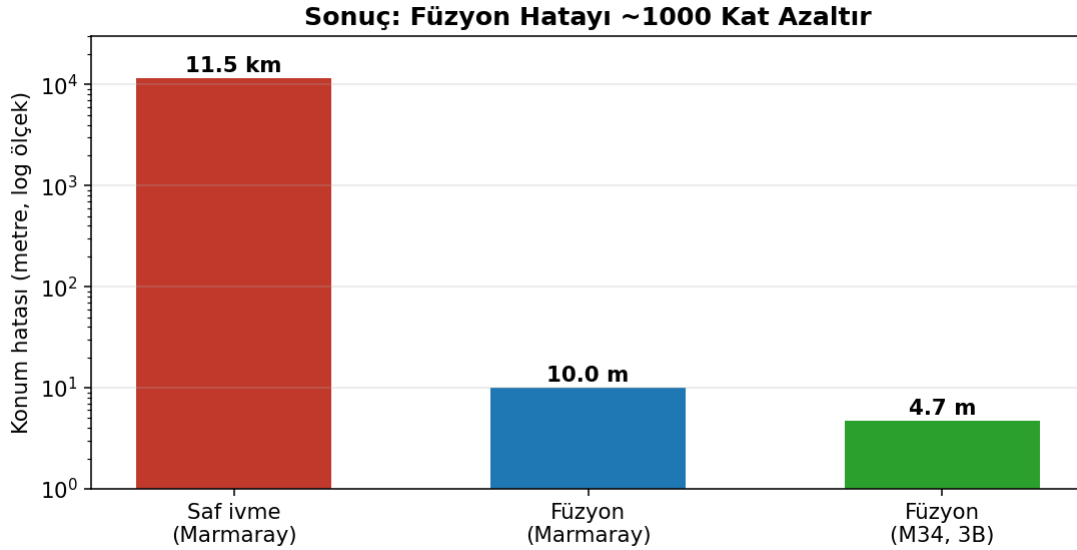
Gerçek bir Marmaray yolculuğunda (36 dakika, 23.8 km) iki yöntemi karşılaştırdık:

Yöntem	Konum hatası
Saf ivme (dead-reckoning)	~11.5 km sürükleniyor ✗
GPS + ivme füzyonu (Kalman)	~10 metre ✓

Ardından telefonla (Samsung M34) yeni saha verisi topladık — bu kez telefonun **üç boyutlu durumu** (eğim/dönüş) da kayıtlıydı:

Ölçüt	Sonuç
Füzyon konum doğruluğu	4.7 metre
10 metre altında kalma oranı	%100

Farkın büyüklüğünü görmek için (log ölçekli) karşılaştırma:

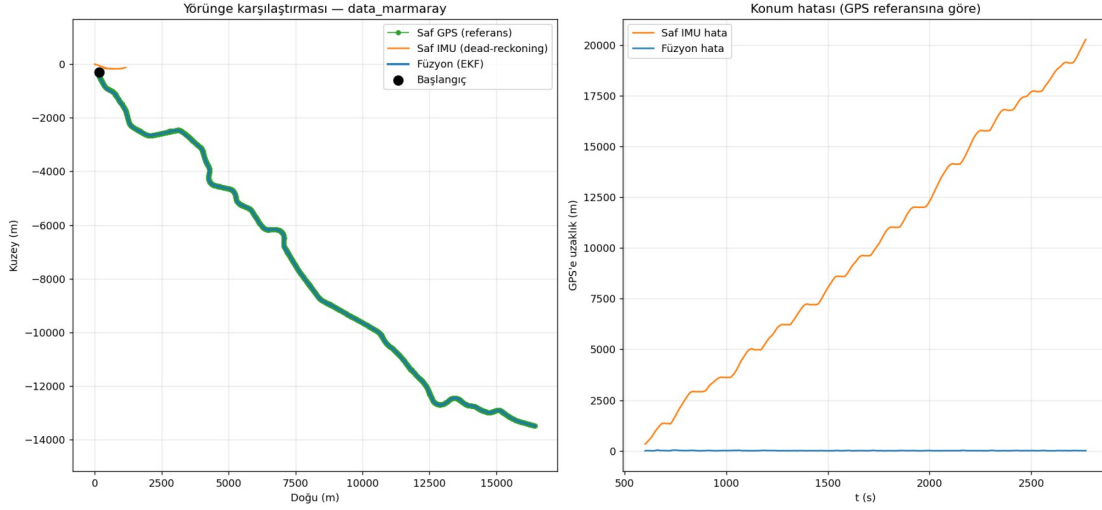


Sonuç: füzyon hatayı yaklaşık 1000 kat azaltır

Gerçek veri grafikleri

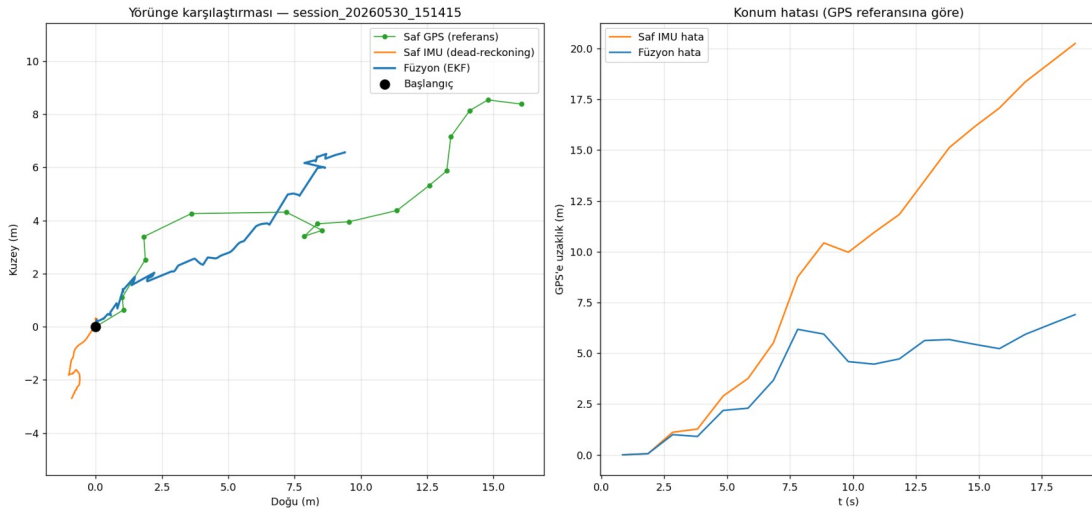
Aşağıda gerçek kayıtlardan üretilen yörünge ve hata grafikleri yer alır. Soldaki panelde yeşil = GPS (referans), turuncu = saf ivme, mavi = füzyon; sağdaki panelde zamanla konum hatası.

Marmaray (36 dk, tünelli):



Marmaray: yörünge ve konum hatası karşılaştırması

M34 saha verisi (3B oryantasyonlu):



M34: füzyon GPS ile neredeyse çakışıyor

💡 **Ders çıkarımı:** Teorik tez (“füzyon şart”) iki ayrı gerçek veri setiyle deneysel olarak doğrulandı. Bilimde döngü budur: **hipotez** → **kod** → **gerçek veri** → **kanıt**.

Ders 4 — Mühendislik Dersleri

Bu çalışma yalnızca teori değildir; gerçek projede karşılaşılan ve öğrencilerin de yaşayacağı derslerle doludur:

- **Tünelde GPS çıldırır.** Marmaray tüneline GPS hatası 1400 metreye fırladı. Çözüm: kötü GPS’i tanıyıp (doğruluk > 50 m ise) reddetmek ve o sırada yalnızca ivmeölçerle “köprü kurmak.”

- **Başlangıç önemlidir.** Filtre, GPS'in ilk *güvenilir* ölçümünde başlatılmazsa, baştaki çöp veriye "emin" başlayıp uzun süre yanlış kalır. Düzeltince maksimum hata 336 m → 37 m'ye düştü.
- **Küçük bir isim hatası tüm programı çökertir.** Bir değişkenin adı değişmişti (bax → bN) ama grafik kısmı eski adı çağırıyordu, program çöküyordu. Tek satırlık düzeltme. Adlandırma tutarlılığı kritiktir.
- **Gizli bilgiyi asla depoya koyma.** Projede bir gizli anahtar (OAuth client_secret) vardı ve yanlışlıkla herkese açık depoya gidebilirdi; yakalayıp engelledik. Güvenlik bir alışkanlıktır.

Tek Sayfalık Özet

İvmeölçer tek başına → hata t^2 ile büyür → ÇÖKER
GPS tek başına → kaba + tünelde kaybolur

KALMAN FÜZYONU = ikisinin en iyisi
hızlı (ivme) + doğru (GPS) + bias'ı öğrenir

SONUÇ: ~11 km hata → ~5 metre

Projenin Geldiği Durum

Bu eğitim notu, gerçek ve çalışan bir projeden (KalmanFilterComparison) üretilmiştir. Projenin bugünkü durumu:

Tamamlananlar

- **Veri toplama:** Marmaray (telefon, 23.8 km) ve sahada Samsung M34 ile 3B oryantasyonlu yeni kayıtlar toplandı. M34 verisi özel yazılan bir Android loglama uygulamasıyla 50 Hz'de toplanıyor.
- **Filtre kütüphanesi:** 8 farklı yöntem karşılaştırmalı olarak kodlandı — KF, Sage-Husa KF, Strong Tracking Filter, EKF ve türevleri, saf dead-reckoning ve ana çözüm **fusionEKF** (GPS konum + Doppler hız füzyonu, canlı bias kestirimi, ZUPT).
- **Doğrulama:** fusionEKF, hem Marmaray (~10 m) hem M34 (4.7 m, <10 m'de %100) verisinde sayısal olarak kanıtlandı. 3B quaternion ile eğim telafisi ilk kez gerçek veride çalıştı.
- **Sağlamaştırma:** Geçersiz GPS'te çökme giderildi; grafik üretimini bozan değişken-adı hatası düzeltildi; gizli OAuth anahtarının sızması engellendi; eksik kaynaklar tamamlanarak depo baştan sona çalışır hale getirildi.
- **Yayınlama:** Tüm çalışma temizlenip GitHub'a yüklendi; üretilen büyük çıktılar depo dışında tutuldu.

Açık / sıradaki işler

- **MATLAB ile Python tutarsızlığı:** MATLAB tarafındaki fusionEKF (~48 m) ile Python tarafındaki (~10 m) arasında yaklaşık 5 kat fark var; iki uygulama hizalanacak.

- **Daha uzun temiz kayıt:** M34 oturumları kısa; 3B oryantasyonlu uzun bir sürüşle füzyonun uzun-tünel performansı daha iyi gösterilebilir.
- **Daha iyi bias modeli ve eğim telafisi** ile tünel drift'inin daha da azaltılması.

Özet: Füzyon algoritması gerçek veriyle ispatlandı, hesaplama hattı uçtan uca çalışır durumda ve proje paylaşılabılır, yedekli, güvenli bir hâle getirildi. Bundan sonrası iyileştirme ve genişletmedir.

Kaynak proje: KalmanFilterComparison • fusionEKF (GNSS/INS gevşek-bağlı füzyon)